FIT3155: Week 2 Lab Sheet

(Questions are based on Week 1 topic, Z-algorithm)

Objectives: Develop an clear understanding and working implementation of Gusfield's Z-algorithm along with its varied applications.

Conceptual exercises

1. Consider the following string:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
a	b	b	a	b	c	a	b	b	a	b	b	a	b	a	b	c

   Work out (on paper) the mechanics of the Z-algorithm when run on this string. While doing so, amongst others, identify which cases (Case 1, Case 2a, Case 2b) apply when computing $Z[k]$ at positions $k = \{7, 10, 13, 15\}$.

2. For a string $S[1 \ldots N]$ (i.e., reasoning in 1-based indexing), suppose we observed that $Z[2] = Len > 0$. Reason why the values of $Z[k]$ for $3 \leq k \leq Len + 2$ can be resolved without additional explicit character comparisons.

3. Now reason which specific case(s) of the Z-algorithm (Case 1, Case 2a, or Case 2b) apply when computing those $Z[k]$ values.

4. Case 2b of the Z-algorithm (i.e., the case handling the scenario where $k \leq r$ and $Z[k - l + 1] \geq r - k + 1$) can be further decomposed into two distinct cases:

   case 2b.1 $Z[k - l + 1] > r - k + 1$

   case 2b.2 $Z[k - l + 1] = r - k + 1$

   Justify that when $Z[k - l + 1] > r - k + 1$, then one can immediately conclude that $Z[k] = r - k + 1$ without needing any explicit character comparisons. In other words, explicit character comparisons are only needed for $Z[k - l + 1] = r - k + 1$ case.

5. Prove that the Z-algorithm runs in $O(N)$ time requiring $O(N)$ space in the worst case, where N is the length of the input string.

6. Given two strings of the same length $S[1 \ldots N]$ and $T[1 \ldots N]$ we want to determine if S is a cyclic rotation of T (or vice versa). How would we approach solving this problem and what is its worst-case time complexity?

7. Write pseudocode for an algorithm that takes two strings $S[1 \dots M]$ and $T[1 \dots N]$ and returns the length of the longest suffix of S that exactly matches a prefix of T.* What is the worst-case time complexity of your algorithm?

8. Given a string $S[1 \dots N]$, write pseudocode for an algorithm that computes an array $Z^{\texttt{suffix}}[1 \dots N]$, where each $Z^{\texttt{suffix}}[i]$ stores the length of the **longest** substring of S **ending** at position i that matches a **suffix** of S of the same length.* Reason the worst-case time complexity of your algorithm.

9. Given a string $S[1 \dots N]$, write pseudocode for an algorithm that computes an array $\texttt{MP}[1 \dots N]$, where each $\texttt{MP}[i]$ stores the length of the largest suffix of the string $S[i \dots N]$ that matches the prefix of S.* Reason the worst-case time complexity of your algorithm.

Implementation exercises

(A message of caution: 'Vibe coding' with GenAI will hinder your learning of algorithms and data structures. The degree (and this unit) you have enrolled into is a specialist degree (unit). By definition it expects one to demonstrate specialist understanding of topics. This understanding comes from making a concerted effort, via thinking, implementing, debugging etc.)

1. In preparation for implementing the Z-algorithm and its application to pattern matching, think and discuss whether there are practical optimisations that your implementation could benefit from, and how these optimisations (if at all) affect the practical space usage and running time, on average and in the worst case.

2. Implement the Z-algorithm. Your code should accept a string as input and return the array of Z-values for the input string.

   Note: In your implementation, use 0-based indexing. The lecture slides present the algorithm using 1-based indexing, quite intentionally. This was mainly to encourage careful reasoning – which is the only way one understands any algorithm – and avoid copying directly without crystallizing the logic for yourself. Therefore, pay very close attention on how to adjust the indexes correctly when implementing the algorithm to avoid errors.

3. Implement the Z-algorithm based exact pattern matching discussed in lectures. Your code should accept a text and a pattern as inputs and return all positions in the text where the pattern matches exactly.a

```
-=0=-
 END
-=0=-
```

*Assume `zalg` is a built-in function available for you, that takes a string as an argument and returns an array of its Z-values.